Advanced Machine learning Mastering Course

Introduced by

George Samuel

Master in computer science
Cairo University

Innovisionray.com

2026

Advanced Machine Learning
2026 by
George Samuel

Visualization

Plotly

```
pip install plotly
```

```
import plotly.express as px
```

```
import plotly.graph_objects as go
```

- **Plotly Express (px):** Think of it as the "Easy Mode." It is a high-level wrapper that allows you to create entire charts in a single line of code.
- **Graph Objects (go):** Think of it as the "Professional/Custom Mode ".
  It is low-level, meaning you have to define every part of the chart manually, but you have total control over every pixel.

Titanic Dataset

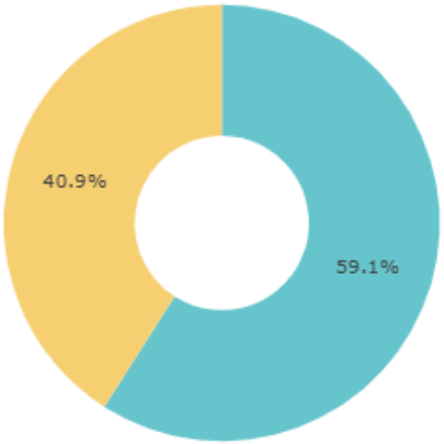
	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Cabin	Embarked
0	0	3	male	22.0	1	0	7.2500	NaN	S
1	1	1	female	38.0	1	0	71.2833	C85	C
2	1	3	female	26.0	0	0	7.9250	NaN	S
3	1	1	female	35.0	1	0	53.1000	C123	S
4	0	3	male	35.0	0	0	8.0500	NaN	S

```
(function) def pie(
    data_frame: Any | None = None,
    names: Any | None = None,
    values: Any | None = None,
    color: Any | None = None,
    facet_row: Any | None = None,
    facet_col: Any | None = None,
    facet_col_wrap: int = 0,
    facet_row_spacing: Any | None = None,
    facet_col_spacing: Any | None = None,
    color_discrete_sequence: Any | None = None,
    color_discrete_map: Any | None = None,
    hover_name: Any | None = None,
) -> ()
```

Data Frame

Column Name

Px.pie



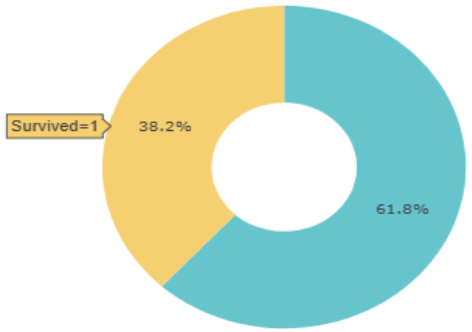
The `px.pie()` function creates a pie or donut chart from categorical data, where `names` defines the slices, `values` controls their size, colors are handled using discrete sequences, and layout options customize appearance.

Plotly

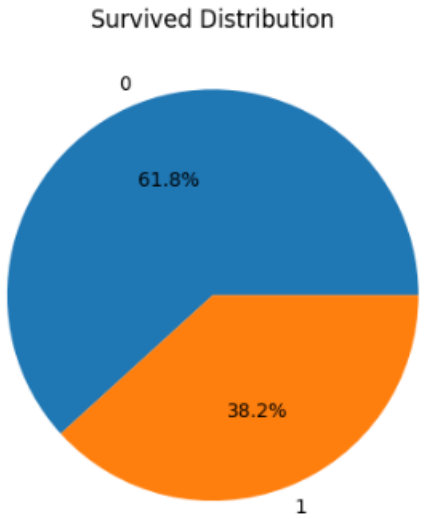
```
fig = px.pie(df,
       names='Survived',
       title='<b>Survived Distribution</b>',
       color_discrete_sequence=px.colors.qualitative.Pastel,hole=0.4)
fig.show()
```

Matplotlib

```
import matplotlib.pyplot as plt
counts = df['Survived'].value_counts()


plt.pie(
    counts.values,
    labels=counts.index,
    autopct='%1.1f%%'
)
plt.title('Survived Distribution')
plt.show()
```



Feature	Plotly
Hover Info	✅ Yes
Click Events	✅ Yes
Zoom / Pan	✅ Yes
Tooltips	✅ Yes

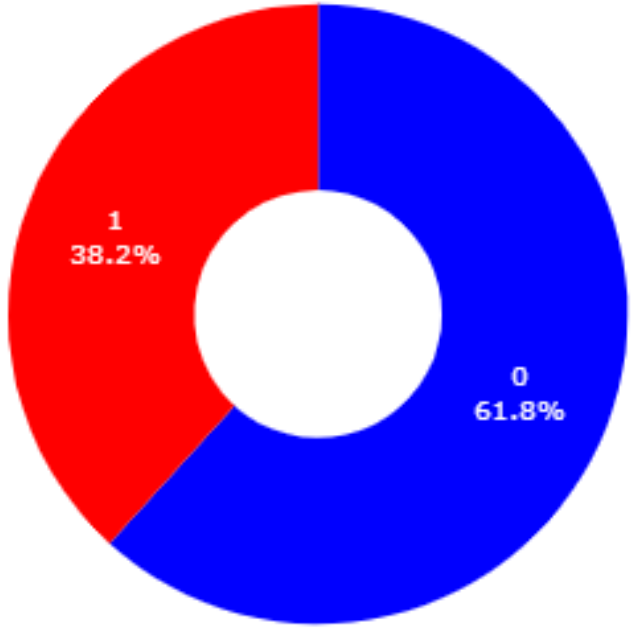


Argument	Value Used	Type	Detailed Explanation
`data_frame`	`df`	Pandas DataFrame	The dataset used to build the pie chart. Plotly reads data directly from this DataFrame.
`names`	`'Survived'`	Column name (str)	Specifies the column whose **unique values** define the pie slices. Plotly automatically counts how many times each value appears.
`title`	`'<b>Survived Distribution</b>'`	str (HTML allowed)	Sets the chart title. HTML tags like `<b>` make the title bold.
`color_discrete_sequence`	`px.colors.qualitative.Pastel`	List of colors	Defines the color palette used for categorical data. `Pastel` provides soft, visually pleasing colors.
`hole`	0.4	float (0 → 1)	Creates a **donut chart** by adding a hole in the center. `0.4` means 40% of the radius is empty.
`fig`	Returned object	Plotly Figure	Stores the generated chart object so it can be displayed or modified later.
`fig.show()`	—	Function call	Renders and displays the chart in the output (Notebook, browser, VS Code, etc.).

Another Attributes

`values`	Uses numerical values instead of automatic counting
`labels`	Renames categories (e.g., 0 → "Died", 1 → "Survived")
`hover_data`	Shows extra info when hovering
`color`	Assigns colors based on a column

```python
fig = px.pie(
    df,
    names="Survived",  # column for pie slices
    title="<b>Survived Distribution</b>",
    hole=0.4,
    color="Survived",  # needed for mapping
    color_discrete_map={
        0: "blue",   # integer keys if column is int
        1: "red"
    },
    category_orders={'Survived':['0','1']}
)

fig.update_traces(textinfo="percent+label")

fig.show()
```

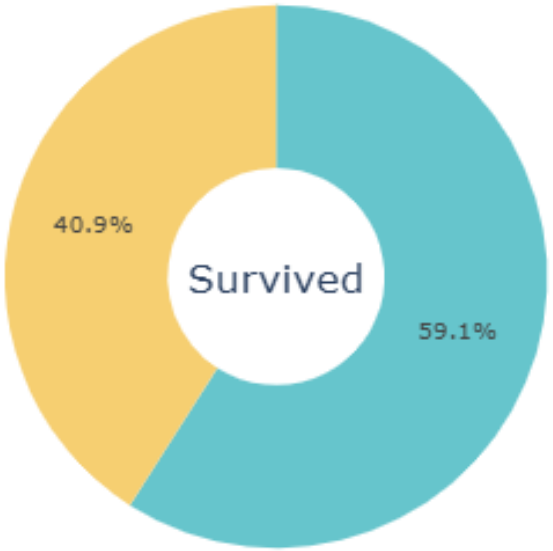


Argument	Value	Meaning
`update_traces()`	—	Modifies properties of the chart slices after creation
`textinfo`	`"percent+label"`	Displays **percentage + category label** on each slice

This code creates a donut chart showing the survival distribution and adds a centered annotation using the layout configuration.

```
px.colors.qualitative.Pastel
[27]  ✓ 0.0s
...  ['rgb(102, 197, 204)',
      'rgb(246, 207, 113)',
      'rgb(248, 156, 116)',
      'rgb(220, 176, 242)',
      'rgb(135, 197, 95)',
      'rgb(158, 185, 243)',
      'rgb(254, 136, 177)',
      'rgb(201, 219, 116)',
      'rgb(139, 224, 164)',
      'rgb(180, 151, 231)',
      'rgb(179, 179, 179)']
```

```python
fig = px.pie(df,
             names='Survived',
             title='<b>Survived Distribution</b>',
             color_discrete_sequence=px.colors.qualitative.Pastel,hole=0.4)
fig.update_layout(annotations=[dict(text= 'Survived', font_size = 20,x=0.5,y=0.5,showarrow= False)])
```

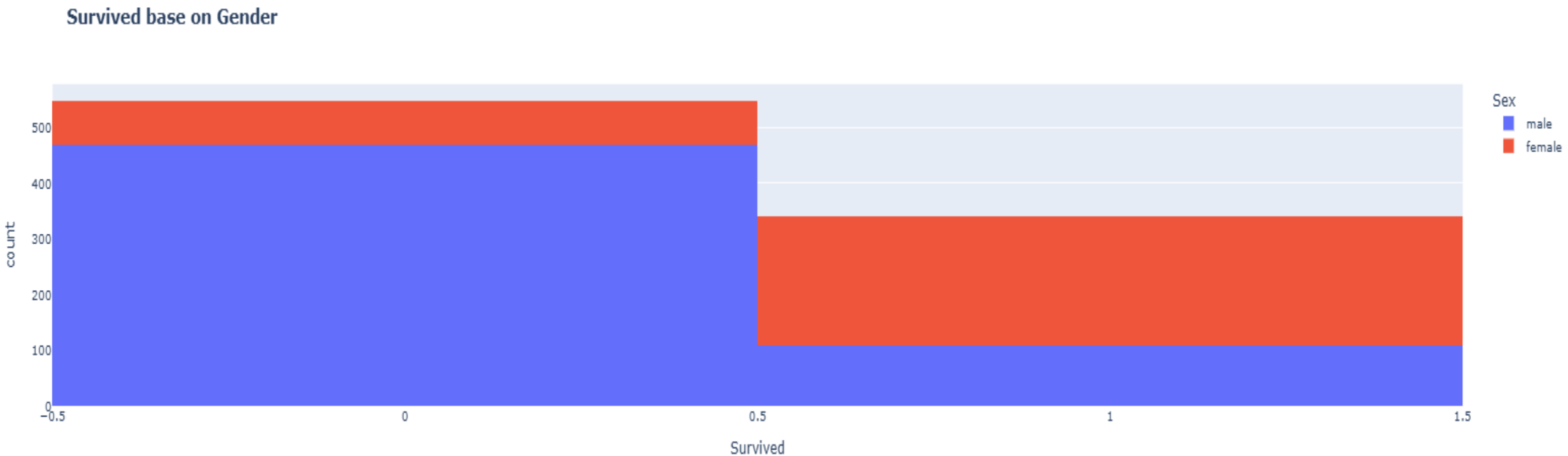


Function	Role
`fig.update_layout()`	Modifies the **layout** of the figure after it is created (titles, annotations, margins, legend, etc.).

```
fig = px.pie(df,
             names='Survived',
             title='<b>Survived Distribution</b>',
             color_discrete_sequence=px.colors.qualitative.Pastel,
             hole=0.7)
fig.update_layout(annotations=[dict(text= 'Survived', font_size = 20,x=0.5,y=0.5,showarrow= False)])
```

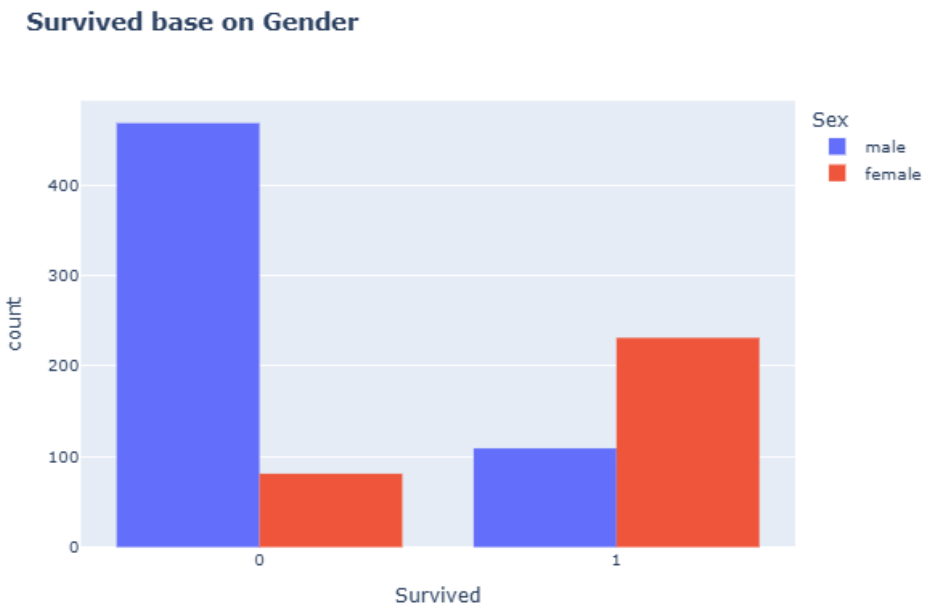
This histogram shows the distribution of survived and non-survived passengers, stacked by gender to compare survival rates between males and females.

```
px.histogram(df,x='Survived',color='Sex',title='<b> Survived base on Gender')#stacked
```



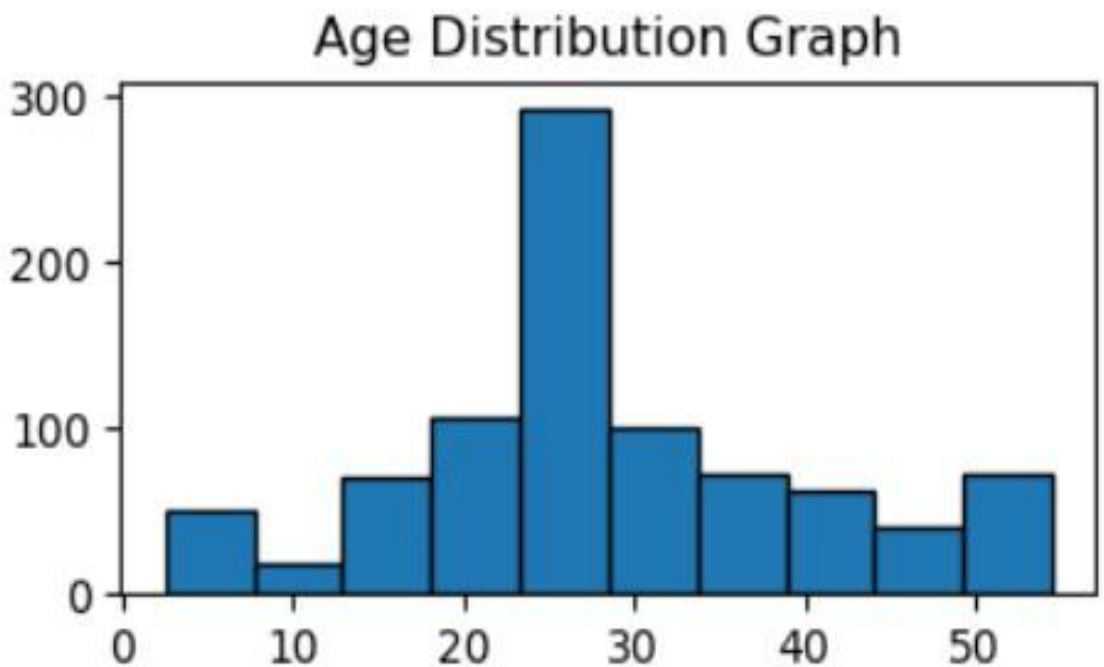
his code creates a grouped histogram that compares survival counts between males and females, with customized figure size and spacing.

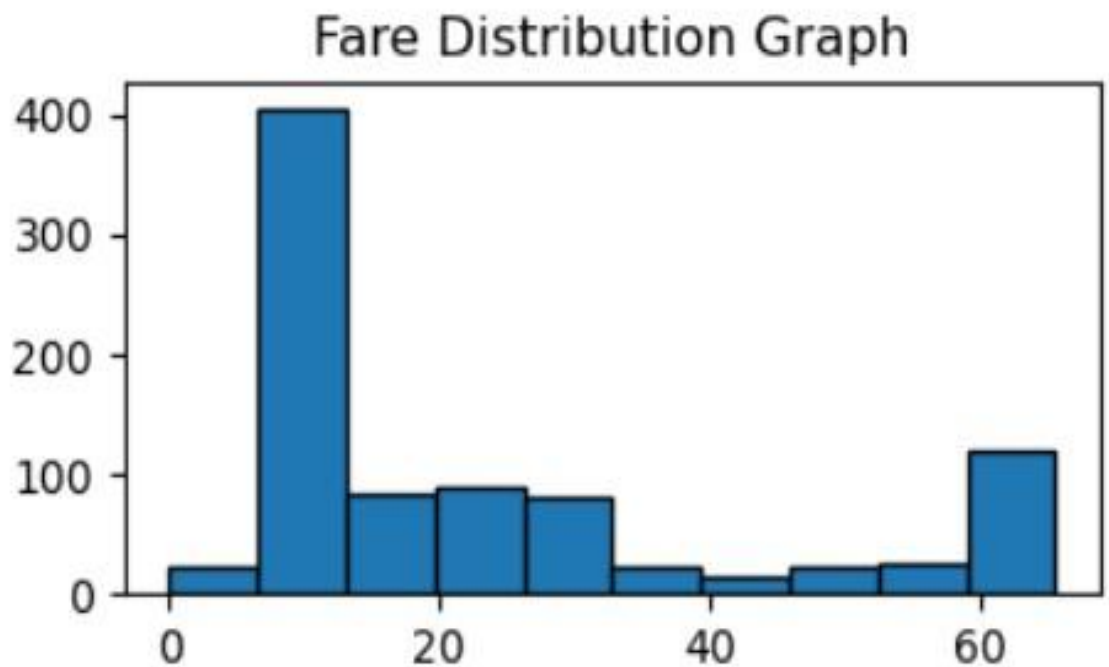
```python
# px.histogram(df,x='Survived',color='Sex',title='<b> Survived base on Gender',barmode='group')
# if you need to update layout you can used it here
fig = px.histogram(df,x='Survived',color='Sex',title='<b> Survived base on Gender',barmode='group')
fig.update_layout(width=700,height=500,bargap= 0.2)
fig.show()
```



barmode	Result
`'stack'`	Bars stacked on top of each other
`'group'`	Bars shown side-by-side ✅
`'overlay'`	Bars overlap
`'relative'`	Positive/negative stacking

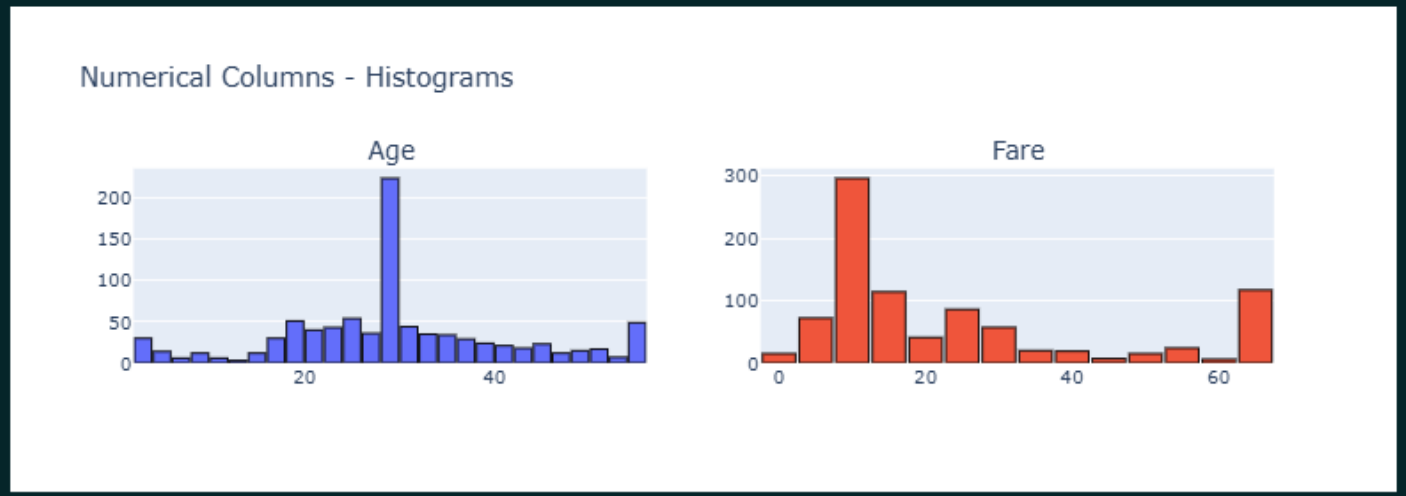
```python
num_cols = df.select_dtypes("number").columns
plt.figure(figsize=(9, 2))
for i, col in enumerate(num_cols):
    plt.subplot(1, 2, i+1)
    plt.hist(df[col], edgecolor="black")
    plt.title(f"{col} Distribution Graph")
plt.show()
```





This code creates multiple histograms for numerical columns using Plotly subplots, allowing easy comparison of data distributions in a single figure.

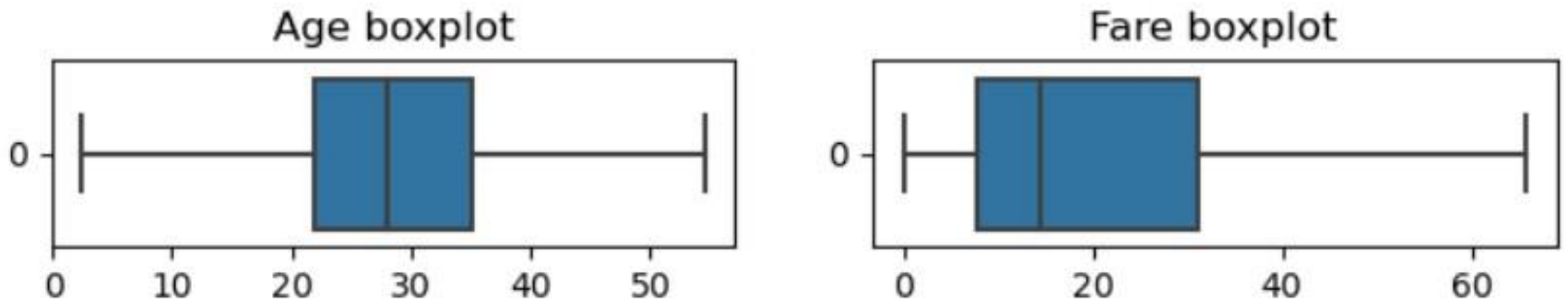
```python
import plotly.subplots as sp
import plotly.graph_objects as go

# Create subplot layout: 1 row × 2 columns
fig = sp.make_subplots(rows=1, cols=len(num_cols), subplot_titles=num_cols)

for i, col in enumerate(num_cols):
    fig.add_trace(
        go.Histogram(x=df[col], name=col, marker=dict(line=dict(color="black", width=1))),
        row=1, col=i+1
    )

# Layout adjustments
fig.update_layout(
    height=300, width=900,
    title_text="Numerical Columns - Histograms",
    showlegend=False,
    bargap=0.1
)

fig.show()
```



B. Outlier Detection Graphs:

- The easiest way to check for outliers, is through displaying a graph called box plot.

```
num_cols = df.select_dtypes("number").columns
plt.figure(figsize=(8, 1))
for i, col in enumerate(num_cols):
    plt.subplot(1, 2, i+1)
    sns.boxplot(df[col], orient="h")
    plt.title(f"{col} boxplot")
```



```python
import plotly.subplots as sp
import plotly.graph_objects as go


# Create subplot layout: 1 row × number of numerical columns
fig = sp.make_subplots(rows=1, cols=len(num_cols),
subplot_titles=num_cols)


for i, col in enumerate(num_cols):
    fig.add_trace(
        go.Box(x=df[col], name=col, boxmean="sd",
orientation="h"),  # horizontal boxplot
        row=1, col=i+1
    )


# Layout adjustments
fig.update_layout(
    height=250, width=800,
    title_text="Numerical Columns - Boxplots",
    showlegend=False
)


fig.show()
```

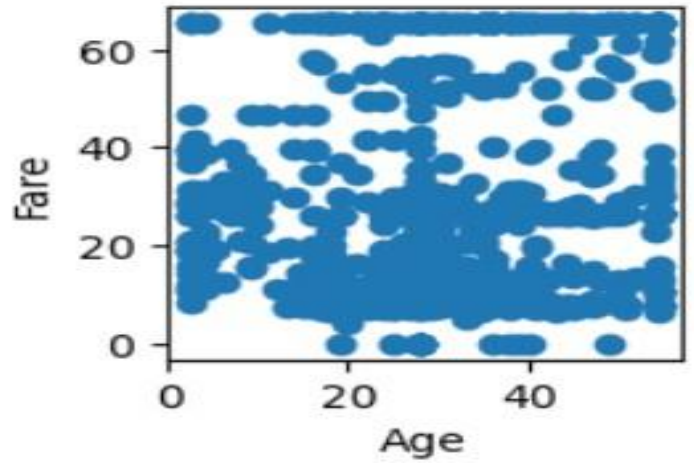
C. Relationship Graphs:

- The idea of this type of graphs is to visualize the relationship between two columns, usually between each feature & the target.
- The dtypes of the two columns defines which graph to use, for example:
  - If the two columns are numerical, we could use scatter plot, pair plot, line plot, or heat map.
  - If we have one numerical & one categorical, then we use bar plot.
  - If the two columns are categorical, then we use heat map.

Numerical/Numerical Relationship (Scatter Plot):

- Get the cartesian coordinates for all the data points in 2d space, where the two dimensions are the two columns.

```
1  plt.figure(figsize=(2, 2))
2  plt.scatter(df["Age"], df["Fare"])
3  plt.xlabel("Age")
4  plt.ylabel("Fare")
5  plt.show()
```

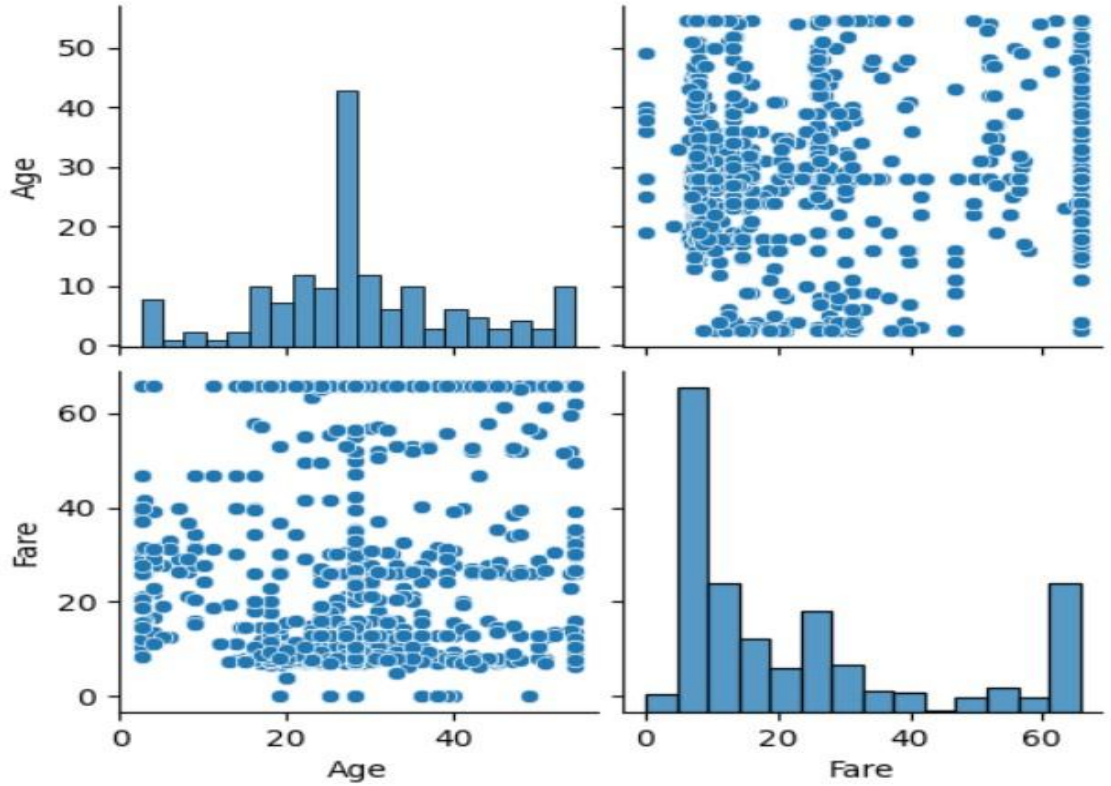


```
fig = px.scatter(
    df,
    x="Age",
    y="Fare",
    title="Age vs Fare Scatter Plot",
    labels={"Age": "Age", "Fare": "Fare"},
    opacity=0.9
)

fig.show()
```

Numerical/Numerical Relationship (Pair Plot):

- Is a graph that gets scatter plot between each two numerical columns.

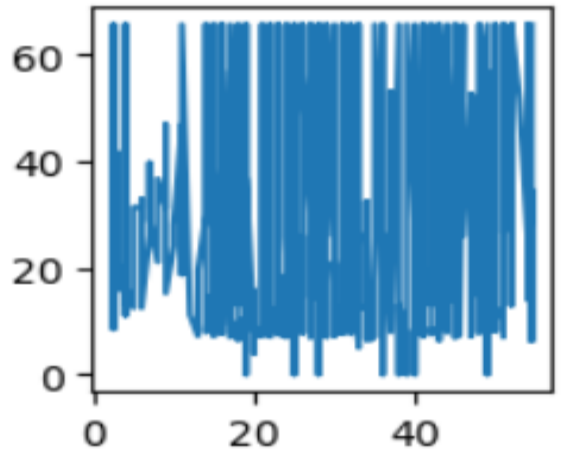
```
1 sns.pairplot(df)
```



Numerical/Numerical Relationship (Line Plot):

- Is the same as scatter plot, but it connect the points with each other with lines, so the data should be sorted first.

```
1  sorted_df = df.sort_values(by="Age")
2  plt.figure(figsize=(2, 2))
3  plt.plot(sorted_df["Age"], sorted_df["Fare"])
4  plt.show()
```

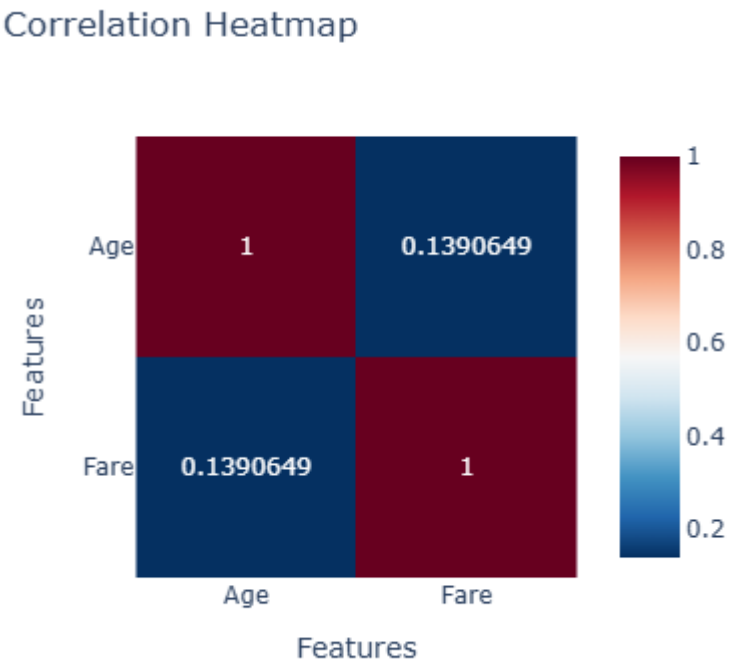


Numerical/Numerical Relationship (Heat Map):

- Is used to show how high values in a 2D-matrix are, this is useful if you want to visualize the correlation matrix of your data.

```
1  corr = df.corr()
2  plt.figure(figsize=(2, 2))
3  sns.heatmap(corr, annot=True)
4  plt.show()
```

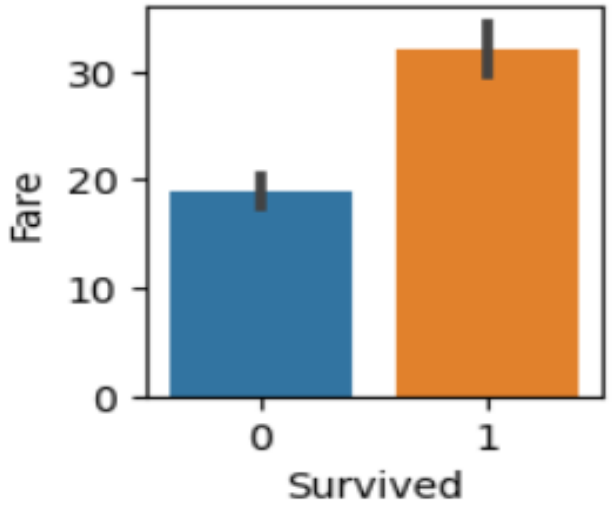
```python
fig = px.imshow(
    corr,
    text_auto=True,    # show correlation values
    color_continuous_scale="RdBu_r",  # red-blue diverging
    title="Correlation Heatmap"
)

fig.update_layout(
    width=400, height=400,
    xaxis_title="Features",
    yaxis_title="Features"
)

fig.show()
```



Numerical/Categorical Relationship (Bar Plot):

- Numerical columns values are aggregated based on the unique values in the categorical column.

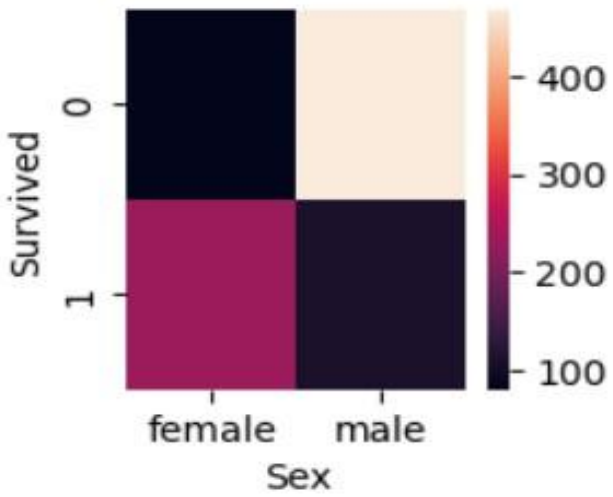
```
1  plt.figure(figsize=(2, 2))
2  sns.barplot(x="Survived", y="Fare", data=df)
3  plt.show()
```



Categorical /Categorical Relationship (Heat Map):

- We first calculate the frequency of each possible pair of unique values form the two columns, then we display the result as a heat map.

```
1  plt.figure(figsize=(2, 2))
2  agg = df.pivot_table(index="Survived", columns="Sex", values="Age", aggfunc=len)
3  sns.heatmap(agg)
4  plt.show()
```



```
# Pivot table
agg = df.pivot_table(index="Survived",
columns="Sex", values="Age", aggfunc="count")


# Interactive heatmap
fig = px.imshow(
    agg,
    text_auto=True,    # show values inside
cells
    color_continuous_scale="Blues",
    title="Counts of Age by Survived and Sex"
)


fig.update_layout(
    width=400, height=400,
    xaxis_title="Sex",
    yaxis_title="Survived"
)


fig.show()


agg
```